

Environnements de développement logiciel



Alexis ROYER - <https://www.linkedin.com/in/alexis-royer/>

2025-2026

Ce document constitue un cours d'ingénierie logicielle, plus particulièrement sur la mise en oeuvre des environnements de développement.

L'environnement de développement contribue à l'efficacité des activités, et par conséquent à la qualité d'un logiciel produit, d'où l'importance de maîtriser l'environnement de travail pour l'ensemble d'une équipe.

Source : <https://github.com/alxroyer/edu-sw-dev-env/>

Ce document est rédigé en format Markdown suivi sous git à dessein, pour démontrer le format Markdown et comment on peut l'utiliser pour la constitution de documentations techniques, ainsi que les capacités de suivi de version et de travail collaboratif apportées par git sur un format texte.

Sommaire :

- 1. Avant-propos
 - 1.1. Types de logiciel
 - 1.2. Technos et langages
 - 1.3. Environnements de développement et qualité logicielle
- 2. Développement
 - 2.1. IDE - Integrated Development Environment

- 2.2. Debugging
- 2.3. Bibliothèques de packages
 - 2.3.1. Fonctionnement
 - 2.3.2. Juridique
 - 2.3.2.1. Licences commerciales
 - 2.3.2.2. Licences Open Source
 - 2.3.2.3. Licences hybrides
 - 2.3.3. Sécurité
 - 2.3.4. Utilisation d'un repo local
- 2.4. Build
 - 2.4.1. Compilateurs / linkers
 - 2.4.2. Bundlers
 - 2.4.3. Gestionnaires de projets
- 2.5. Industrialisation
- 3. Exécution
- 4. Assurance qualité
 - 4.1. Pratiques d'équipe
 - 4.2. Règles de codage
 - 4.3. Versionning / Gestion de configuration
 - 4.3.1. Source control - git
 - 4.3.2. Versioning - SemVer
 - 4.3.3. Ticketing
 - 4.3.4. PLM - Product Lifecycle Management
 - 4.3.5. Archivage
 - 4.3.6. Index de configuration
 - 4.4. Documentation
 - 4.4.1. GED
 - 4.4.2. CMS
 - 4.4.3. Formats texte + git
- 5. Delivery
 - 5.1. Enregistrements
 - 5.2. CI/CD
- 6. Déploiement

1. Avant-propos

1.1. Types de logiciel

Avant toute chose, il convient de considérer que la mise en oeuvre d'un environnement de développement est fortement dépendante du type de logiciel développé.

On distingue différents types de logiciels :

-  les applications PC (Windows, Mac, Linux),

-  les applications web (front et back),
-  les applications mobile (Android, iPhone),
-  les logiciels embarqués.

1.2. Technos et langages

Pour chacun des types de logiciels, des technologies différentes peuvent être mises en jeu (liste non exhaustive) :

-  Applications PC :
 - Langages :
 - C / C++
 - Java
 - Node.js (1) / Typescript (2)
 - Python
 - Go : exemple [esbuild](#) (6)
 - Rust (3)
 - Systèmes d'exploitation :
 - Windows
 - Mac
 - Linux
 - Frameworks :
 - Qt (4)
 - React Native (4)
-  Applications web :
 - Langages :
 - HTML
 - Côté *frontend* :
 - Javascript / Typescript (2)
 - Côté *backend* :
 - Node.js (1) / Typescript (2)
 - Java
 - PHP
 - Ruby
 - Python
 - C / C++
 - Go (6)
 - Frameworks :
 - Angular (Javascript)
 - Vue.js (Javascript)
 - React Native (Javascript) (4)
 - Symfony (PHP)
 - Ruby on Rails

-  Applications mobiles :
 - Architectures / systèmes d'exploitation :
 - Android
 - iOS
 - Langages :
 - Java, Kotlin (Android)
 - Swift (iOS)
 - Objective-C (iOS)
 - C (Code natif Android & iOS)
 - Frameworks :
 - React Native (4)
 - Kits de développement :
 - SDK Android : Software Development Kit
 - NDK Android : Native Development Kit
-  Logiciels embarqués :
 - Langages :
 - C / C++
 - Go ? à confirmer (6)
 - Rust (3)
 - Systèmes d'exploitation :
 - Linux : couramment utilisé pour réaliser des systèmes embarqués. (5)
 - vxWorks : système d'exploitation dit *temps-réel*, propriétaire, développé par Wind River.
 - µC-OS : OS minimaliste, sans système de fichier, pile IP, ... par défaut.
 - Sans OS : simple boucle de réaction sur événement.
 - Frameworks / kits de développement :
 - buildroot

Notes :

- (1) Node.js : extension de Javascript, initialement restreint aux navigateurs, avec exécution par un moteur dédié.
- (2) Typescript : extension de la syntaxe Javascript pour permettre le typage.
- (3) Rust est un langage de programmation compilé, relativement nouveau, dont un des objectifs principaux est la sécurisation de la gestion de la mémoire. De fait, certains le présentent comme le successeur de C / C++.
- (4) Certains frameworks ont l'avantage d'être *cross-platform*, c'est à dire qu'ils permettent de réaliser un seul développement utilisable sur différents environnements (plateformes et/ou systèmes d'exploitation).
- (5) L'OS Linux, même dans une version embarquée, n'est pas considéré comme un OS *temps-réel*, c'est-à-dire qu'il ne garantit pas un temps d'exécution pour un traitement donné. L'occupation mémoire est plus importante que pour un OS minimaliste. En revanche, il a l'avantage de fournir un écosystème riche de services et d'outils qui peut faciliter les développements.
- (6) Go plebiscité pour ses capacités de gestion multi-thread, qui peut avoir son intrêt pour le développement d'applications avec des contraintes de perfs. Langage un peu spécifique, qui

nécessite une phase d'apprentissage spécifique.

Le choix du langage et des technologies peut dépendre de plusieurs facteurs :

- en premier lieu, de la nature du logiciel développé,
- mais aussi des caractéristiques attendues : performances, empreinte mémoire, look & feel, ...
- de la culture d'entreprise : compétences disponibles en interne,
- de la stratégie d'entreprise : volonté d'investir sur telle ou telle technologie, ou de rationaliser les technologies utilisées, ...
- de la disponibilité des compétences sur le marché de l'emploi : capacité à staffer des équipes, assurer la maintenance dans le temps, ...
- ...

Quoi qu'il en soit, le langage et les technologies retenus conditionnent fortement l'environnement de développement à mettre en place.

i Vidéos pour le fun

Most popular coding language 2012 - 2024 : <https://www.youtube.com/watch?v=viPKPhOUcyU>

Most Popular Programming Languages 1955 - 2025 : https://www.youtube.com/watch?v=5yAbVkiML_M

1.3. Environnements de développement et qualité logicielle

La norme ISO 9126, ainsi que l'ISO 25010 qui la remplace aujourd'hui, identifient différentes dimensions pour la caractérisation de la qualité d'un logiciel. Entre autres (1) :

- Capacité fonctionnelle
- Facilité d'utilisation
- Fiabilité
- Performance
- Maintenabilité
- Portabilité

Une partie de ces dimensions reste essentiellement portée par le logiciel développé.

Toutefois, l'environnement de développement peut contribuer à certaines de ces dimensions, notamment :

- Fiabilité :
 - Mettre en oeuvre des outils d'analyse de code statique, permettant d'éviter des bugs avant même d'exécuter le code.
 - Exécuter régulièrement des tests de non-régression, pour détecter les problèmes au plus tôt.
- Maintenabilité :
 - Assurer un maximum de confort pour les développeurs, de sorte à faciliter le codage, le debugging... de par le choix des outils, pour leurs fonctionnalités, et sans oublier leurs performances ! Faire en sorte de limiter les freins aux différentes actions nécessaires à la réalisation d'un logiciel de qualité.
 - Faciliter notamment les refactorings, nécessaires à la maintenabilité.

- Utiliser un outil de versioning (git).
- Automatiser les vérifications de qualité (linters, indentation) pour s'assurer d'un certain niveau de lisibilité du code, donc un code plus facile à maintenir, à suivre en historique.
- Portabilité :
 - Exécution des tests de non-régression sur les différentes cibles.

i (1) Accès aux normes ISO

La difficulté des normes ISO est que celles-ci ne sont pas libres d'accès.

En effet, le site de référence de l'ISO ne fournit qu'une prévisualisation de la version en cours de la norme ISO 25010 : <https://www.iso.org/obp/ui/#iso:std:iso-iec:25010>

To view the full content, you will need to purchase the standard by clicking on the "Buy" button.

Toujours sur le site de référence, difficile de trouver une telle prévisualisation pour l'ISO 9126.

La liste simplifiée ci-dessus est tirée de la page Wikipédia
https://fr.wikipedia.org/wiki/Qualit%C3%A9_logicielle

2. Développement

Avant de délivrer et exécuter le logiciel, commençons par mettre en oeuvre les éléments permettant de le construire.

2.1. IDE - Integrated Development Environment

Un des premiers outils qu'on est amené à aborder pour faire du développement logiciel est l'IDE.

L'usage d'un IDE présente de nombreux intérêts :

- Edition de code : Basique, un simple notepad (voire **vi** pour les puristes !) pourrait faire le job.
- Coloration syntaxique : Ca change déjà un peu la vie, dans la mesure où ça fluidifie la lecture du code.
- Navigation : Capacité à aller directement sur le code d'une variable ou d'une fonction, ou dans le code Markdown d'un chapitre donné. Possibilité de gagner du temps, en évitant de scroller et de faire des CTRL+F dans tous les sens quand le code commence à prendre du volume.
- Complétion : La complétion apporte un vrai plus, en évitant des fautes de frappe donc des erreurs chronophages au passage.

⚠ Attention : Complétion et fautes d'orthographe

Il existe un risque avec la complétion de répliquer des fautes d'orthographe ou des dyslexies sur les noms de variables ou de fonctions dans la totalité de code.

Le cas échéant, avoir le réflexe d'utiliser les fonction de refactoring pour corriger le tir.

- Refactoring : On gagne encore plus du temps. On n'hésite plus à renommer une variable, une fonction, une classe, pour lui redonner un nom plus adapté, allant dans le sens de la maintenabilité.

- Edition multi-lignes : Capacité mise à disposition par un certain nombre d'éditeurs de texte, à laquelle on s'habitue très vite dès lors qu'on commence à l'utiliser.
- Debugging : Dès lors que c'est possible, permet d'obtenir plein d'informations utiles dans le cadre de résolution d'un bug. Possibilité également d'utiliser les fonctions de debugging pour tester des injections de fautes.
- IA : Les IDE intègrent aujourd'hui des fonctions d'IA qui peuvent augmenter la productivité.

⚠ Attention : IA et confidentialité

A ce jour, l'IA fonctionne principalement par communication d'informations sur des moteurs s'exécutant sur Internet.

L'utilisation de l'IA dans les IDE peut donc être problématique si l'entreprise ne souhaite pas divulguer le code.

Bien se renseigner sur les usages possibles ou non.

Possible que l'entreprise dispose d'un contrat de confidentialité avec une IA donnée, voire héberge une solution d'IA *on premise*, auquel cas on pourra configurer les IDE pour utiliser cette IA validée par l'entreprise.

On liste ci-après des IDE connus (liste non exhaustive) :

-  Applications PC :
 - Visual C++
 - Eclipse : Java, mais aussi C / C++ (version CDT)
 - VS Code : Node.js, Typescript, mais aussi C / C++ (extension)
 - JetBrains IntelliJ IDEA : Java
 - JetBrains PyCharm : Python
-  Applications web :
 - VS Code : HTML, Javascript, Typescript, PHP
 - PhpStorm
 - JetBrains IntelliJ IDEA : Java
-  Applications mobile :
 - Android Studio
 - JetBrains IntelliJ IDEA : Java, Kotlin
 - XCode : iOS
-  Logiciel embarqué :
 - VS Code : C / C++ (extension), Rust (extension)
 - Eclipse : C / C++ (version CDT)

Le tableau ci-après donne un aperçu historisé de l'apparition de certains des IDE précédemment cités :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

IDE	Langages	Description	Historique
Turbo Pascal	Pascal	L'un des premiers IDE modernes.	1983
Visual Studio	C, C++, C#	Développé par Microsoft.	1997
NetBeans	Java, PHP, C/C++	IDE polyvalent, version Open Source.	2000
Eclipse	Java, C++ (plugin CDT, 2004)	IDE open source, pour Java, en Java, extensible par plugins, réputation de lourdeur.	2001
IntelliJ IDEA	Java, ...	IDE populaire, développé par JetBrains, extensible par plugins.	2001
Xcode	C, Objective-C, Swift	IDE d'Apple pour le développement macOS et iOS.	2003
PyCharm	Python	Egalement développé par JetBrains.	2010
VS Code	HTML, CSS, JS/Typescript, Markdown, ...	Éditeur de code léger et extensible, développé en JS par Microsoft, écosystème riche de plugins.	2015



TP : Configuration d'un IDE VS Code pour C/C++



TP : Refactoring (TODO)

2.2. Debugging

i Environnements d'exécution

On parle de debugging dès à présent, car c'est une des fonctions majeures apportées par un IDE facilitant l'investigation et la résolution de bugs logiciels.

Cela entend qu'on sait exécuter notre logiciel au préalable (cf. § Exécution). La notion reste toutefois abordable en considérant le cas simple des applications PC dans un premier temps.

Les debuggers (tels `gdb`) peuvent généralement s'utiliser en ligne de commande. Toutefois, cela reste extrêmement compliqué, et l'utilisation d'une interface graphique est quasiment indispensable pour déboguer efficacement.

Les IDE intègrent très bien les débogueurs, et permettent ainsi de basculer très rapidement entre les modes édition de code et debugging / intégration.

Parmi les fonctions utiles d'un debugger, on note :

- la possibilité de positionner des points d'arrêt (*breakpoints*)
- l'exécution pas à pas, i.e. ligne à ligne,
- l'observation des valeurs de variables,
- rentrer dans l'exécution des fonctions (et en ressortir)
- l'affichage de la pile d'appels (*callstack*)

- la possibilité de définir un point d'arrêt sur modification de valeur : très utile pour investiguer des problèmes de débordement mémoire
- la capacité à modifier des valeurs de données : utile pour tester des injections de fautes
- la capacité pour certains langages interprétés (Python, JS) à pouvoir exécuter des routines à chaud
- ...



TP : Debugging C/C++ avec VS Code

Dans le cas du debugging d'un logiciel embarqué, la difficulté est de savoir synchroniser un IDE/debugger qui tourne sur une machine de développement, et le programme à débbugger qui tourne sur une cible autre. Pour ce faire, on peut utiliser des moyens spécifiques :

- **gdbserver** :
 - Possible pour des cibles tournant sous Linux uniquement.
 - Le programme à débbugger est lancé sur la cible avec **gdbserver**, et le debugger sur la machine de développement se connecte au **gdbserver** en tant que client.
 - J'ai déjà constaté personnellement que la connexion pouvait être instable. Pas de garantie notamment que les points d'arrêts soient bien pris en compte, notamment dans des cas d'applicatifs multi-threadés.
- Utilisation d'une sonde :
 - Matériel de sonde spécifique au micro-processeur de la cible.
 - Branchement sur un bus série dédié, directement sur le micro-processeur sur la cible. Nécessite que les signaux aient été routés sur la carte.



TP : Debugging C/C++ à distance avec gdbserver (TODO)

On note finalement que les investigations par debugging ont leurs limites :

- Pas de debugging possible *post-mortem*, i.e. après constat d'un problème après que l'exécution du logiciel soit terminée (cas typique d'une exception ou d'un crash constaté en prod) et tant qu'on n'a pas caractérisé les conditions de reproduction du problème.
- Il arrive fréquemment que le fait de poser des points d'arrêts provoque des effets de bords sur des timeouts qui arrivent à échéance, et donc des effets indésirables empêchant un debugging confortable.
- Dans le cas des logiciels embarqués, comme le logiciel s'exécute sur une cible autre, le debugging est par nature moins facile à mettre en oeuvre.
- Cas en Python où le débbugger exécute automatiquement des routines **__repr__()** pour obtenir une représentation des objets à afficher dans l'IHM de debug, ces exécutions surnuméraires pouvant provoquer des effets de bords indésirables selon les cas.

En raison de ces limitations, la bonne vieille technique du debugging "à la trace" ne doit pas être négligée.

Plus encore, il convient de mettre en place un système de logs efficace dès le début du projet, avec quelques fonctions utiles :

- Niveaux de criticité : debug, info, warning, erreur classiquement.
- Colorisation : savoir faire apparaître les erreurs en rouge a minima.

- Indentation : utile pour analyser les logs de traitements récurifs.
- Filtrages : par fonction, par module.

L'utilisation d'outils tels que Kibana peut également s'avérer utile pour exploiter ces logs.

2.3. Bibliothèques de packages

2.3.1. Fonctionnement

Aujourd'hui, on écrit rarement l'intégralité du code d'un logiciel. On repose généralement sur un écosystème riche de frameworks et de librairies.

Ces écosystèmes de frameworks ou librairies s'organisent généralement en deux parties :

- une *registry* en ligne : un site gérant une bibliothèque de librairies.
 - Ces sites permettent de naviguer dans la bibliothèque de librairies, faire des recherches, ...
 - Chaque librairie est documentée : auteur, versions, dépendances, ...
 - On peut télécharger chacune des librairies pour les installer *à la main*.

i Packages

Les frameworks ou librairies pouvant être constitués de différents éléments (code source, licence, documentation, scripts utiles, ...), le téléchargement est facilité par le conditionnement dans une archive unique.

On nomme généralement cette archive un *package*.

Certaines technos affectent un nom spécifique pour ces packages, comme Rust qui nomme ses packages des *crates*.

- un programme s'exécutant en local, permettant de :
 - gérer un projet : build, debug, test, ... c'est l'équivalent de ce qu'on peut faire *à la main* avec un Makefile (cf. § [Gestionnaires de projets](#)),
 - déclarer les packages qu'on souhaite utiliser au travers du projet géré par l'outil,
 - s'interfacer avec la *registry*,
 - résoudre les dépendances des librairies.

Il existe différents systèmes de packages, généralement centrés sur une technologie donnée.

On liste ci-après des systèmes de packages usuels (liste non exhaustive) :

-  Applications PC :
 - JS : npm / yarn / pnpm (1)
 - Java : maven
 - Python : pip / venv
-  Applications web :
 - JS : npm / yarn / pnpm (1)
 - PHP : composer

- Java : maven
-  Applications mobile :
 - Android : repo (2)
 - iOS : **swift package**
-  Logiciel embarqué :
 - C/Linux : buildroot (3)
 - Rust : Cargo

Le tableau ci-après donne un aperçu historisé de l'apparition de certains des systèmes de packages précédemment cités :

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Langage	Outil projet	Registry	Historique
Perl	cpan	https://metacpan.org/	1995
C/Linux	buildroot (3)	https://buildroot.org/	2001
Java	mvn V1 (2002), V2 (2008)	https://search.maven.org/ (2005)	2002
Python	pip, venv	https://pypi.org/	2003
Ruby	gem	https://rubygems.org/	2004
JS	npm, yarn (2016), pnpm (2017)	https://www.npmjs.com/	2010
PHP	composer	https://packagist.org/	2012
Go	go get	https://pkg.go.dev/	2012
Rust	cargo	https://crates.io/	2014
Swift	swift package	https://www.swift.org/package-manager/	2016

i Notes

- (1) yarn et pnpm constituent des améliorations de npm.
 - yarn, d'après <https://yarnpkg.com/> : "Safe, stable, reproducible projects"
 - pnpm, d'après <https://pnpm.io/> : "Save time. Save disk space. Supercharge your monorepos."

J'ai effectivement pu constater personnellement que npm pouvait avoir des limitations dans des cas particuliers.

C'était notamment le cas pour la gestion de différentes versions d'un même package (en l'occurrence React Native) pour les différentes targets dans un monorepo (cf. issue [npm#287](#)).

En raison de ces difficultés, j'étais passé sur yarn.

- (2) **repo** n'est pas vraiment un système de packages, mais plutôt une extension de git permettant de manipuler un grand nombre de dépôts dans l'espace projet.

Il n'y a pas vraiment de *registry* en ligne pour **repo**, mais simplement des dépôts git.

- (3) buildroot n'est pas vraiment centré sur un langage, mais sur des ensembles de bibliothèques et logiciels pouvant être embarqués dans l'image Linux constituée.

En un sens, cela ressemble plus à des systèmes de packages pour l'OS Linux, comme rpm (Red Hat Package Manager pour l'origine du nom) ou apt/dpkg (packages Debian .deb), à la différence que les packages ne sont pas téléchargés sous forme de binaires, mais de sources pour être cross-compilés.

On retrouvera classiquement pour Linux beaucoup de code C, mais pas uniquement.

  TP : API REST en JS avec npm

  TP : API REST en Python avec pip et venv (TODO)

  TP : API REST en PHP avec composer (TODO)

  TP : API REST en Rust avec cargo

  TP : Image Linux avec buildroot (TODO)

Si le fait d'utiliser des bibliothèques tierces permet de faciliter et d'accélérer les développements, cela ne va pas sans certains inconvénients.

Astuce : Privilégier un disque SSD

Comme on peut le voir, certains projets peuvent être amenés à tirer un volume conséquent de packages.

Les packages étant enregistrés sur le disque local, cela peut représenter beaucoup d'accès disque.

Il peut ainsi être intéressant de privilégier l'utilisation de disques SSD pour éviter des phénomènes de lenteur.

J'ai eu l'occasion de constater un vrai gain avec un disque SSD dans certains cas de figure :

- multiplicité des dépôts git Android tirés avec la commande **repo**,
- volume de packages tirés dans un monorepo JS avec React Native.

2.3.2. Juridique

Toute bibliothèque logicielle vient généralement avec une licence d'utilisation.

Les développeurs ont leur part de responsabilité vis-à-vis des licences tirées en fonction des choix de technologies opérés.

2.3.2.1. Licences commerciales

Il existe différents modes de facturation :

- au nombre de développeurs,
- par famille de produits,
- au runtime, i.e. par unité produite (souvent le plus coûteux),
- ...

Lorsque la licence est commerciale, penser à s'acquitter des droits à payer !

Tout oubli de règlement des licences commerciales peut se payer très cher devant les tribunaux, notamment lorsqu'on intègre les arriérés, plus probablement des pénalités à la clé !

2.3.2.2. Licences Open Source

Attention ! Open Source ne veut pas dire libre de droits.

Il existe différents types de licences Open Source plus ou moins permissives.

Le premier tableau ci-après donne une liste des principales licences dites permissives.

Ces licences permissives restent compatibles avec le développement d'un logiciel propriétaire, à la condition toutefois de se conformer aux termes de la licence.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
MIT (Massachusetts Institute of Technology)	MIT	Utilisation, modification et distribution libres, y compris dans des projets propriétaires. Seule obligation : conserver la notice de copyright et la décharge de responsabilité.	Ruby on Rails, jQuery
ISC (Internet Software Consortium)	ISC	Similaire à MIT, mais avec une formulation plus simple. Proposée par défaut dans l'écosystème JS.	npm
BSD (Berkeley Software Distribution)	BSD-3- Clause , BSD-2- Clause , 0BSD	Similaire à MIT, mais plus précise en termes de limitation de responsabilité du développeur vis-à-vis des usages faits du logiciel. Existe en différentes versions (nombre de clauses).	FreeBSD, NetBSD
Apache 2.0	Apache-2.0	Protection contre les brevets et clause de non-responsabilité. Compatible avec la GPLv3. Obligation de conserver les notices de licence et de ne pas utiliser les marques déposées.	Apache HTTP Server, Apache Kafka, une grande partie de code Android

Le second tableau suivant liste des licences dites *copyleft faible*.

Ce type de licences reste compatible avec le développement d'un logiciel propriétaire, mais impose généralement la reversion des modifications qui pourraient être apportées dans le code Open Source utilisé.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
LGPL (Lesser GPL)	LGPL-3.0-only , ...	Permet l'utilisation dans des logiciels propriétaires, mais les modifications de la bibliothèque LGPL doivent rester open source (i.e. obligation de publication des modifications).	GTK
MPL 2.0 (Mozilla Public License 2.0)	MPL-2.0	Copyleft faible par fichier.	Firefox, Thunderbird

Enfin, le troisième tableau liste des licences dites *copyleft fort* parmi les plus connues.

Ces licences imposent à un logiciel *dérivé* d'adopter lui-même la licence Open Source.

Ce qu'il faut entendre par logiciel *dérivé* c'est entre autres le fait de se *linker* avec le logiciel Open Source.

C'est pourquoi on qualifie ces licences de *contaminantes*.

De fait, une librairie avec ce type de licence est généralement incompatible avec le développement d'un logiciel propriétaire.

Source : <https://chat.mistral.ai/>, sous réserve de confirmation des informations.

Licence	Identifiant SPDX (1)	Principales caractéristiques	Exemples
GPL (GNU General Public License)	GPL-2.0-only , GPL-3.0-only , ...	Toute oeuvre dérivée doit être distribuée sous GPL également. La version en vigueur est la V3, mais on trouve également beaucoup de V2.	Linux, gcc
AGPL (Affero GPL)	AGPL-3.0-only , ...	Similaire à la GPL (licence contaminante), mais étend les obligations aux logiciels utilisés en réseau (SaaS).	itext
SSPL (Server Side Public License) (2)	SSPL-1.0	Licence créée par MongoDB, inspirée de AGPL.	MongoDB, ElasticSearch

i Notes

- (1) [SPDX](#) (System Package Data Exchange), est un standard ouvert permettant de spécifier la décomposition d'un logiciel, via une SBOM (Software Bill of Materials).

Ce standard propose des codification des licences logicielles (cf. <https://spdx.org/licenses/>).

- (2) Controverse : SSPL n'est pas reconnue comme une licence Open Source à part entière par l'OSI (*Open Source Initiative*). Cf. <https://opensource.org/blog/the-sspl-is-not-an-open-source-license>.

SPDX & conformité juridique

Des outils associés au standard SPDX permettent de vérifier la conformité juridique aux licences embarquées.

Cf. <https://spdx.dev/use/spdx-tools/>.

i CC - Licences Creative Commons

Proche des licences Open Source, il existe également les licences Creative Commons.

Ce type de licence est plutôt dédié à la protection d'oeuvres créatives numériques, littéraires ou artistiques.

Cf. <https://creativecommons.org/>.

Note : Ce cours est publié sous licence CC BY-NC-SA. Cf. [LICENSE.txt](#).

2.3.2.3. Licences hybrides

Il arrive régulièrement que des projets Open Source adoptent un *business model* hybride :

- licence commerciale pour les entreprises,
- licence Open Source pour les utilisations personnelles, éducatives ou Open Source.

Exemples :

- Qt : licence [GPL](#) ou [LGPL](#) ou [commerciale](#).
- Elasticsearch : licence [SSPL](#) ou [AGPLv3](#) ou [commerciale](#).
- MongoDB : licence [SSPL](#) ou [commerciale](#).
- itext : licence [AGPL](#) ou [commerciale](#).

2.3.3. Sécurité

Embarquer des librairies tiers signifie également embarquer les bugs que celles-ci contiennent, et pire encore leurs failles de sécurité !

C'est pourquoi il convient de mettre en place une MCS (Maintien en Conditions de Sécurité), c'est-à-dire une veille sur les alertes de sécurité et failles publiées.

Sources d'information pour la mise en oeuvre d'une veille MCS:

- CVE (*Common Vulnerabilities and Exposures*) :
 - Depuis 1999 (première CVE : [CVE-1999-0001](#)).
 - Base de failles de sécurité connues pour les différents logiciels référencés.
 - Plus de 300 000 CVE en base début février 2025.

- Page de recherche : <https://www.cve.org/CVERecord/SearchResults> (redirection depuis <https://cve.mitre.org/>).
- CERT-FR (French Computer Emergency Response Time) de l'[ANSSI](#) (Agence Nationale de la Sécurité des Systèmes d'Information) :
 - <https://www.cert.ssi.gouv.fr/>

Pour mener les analyses d'impact liées à l'occurrence d'une alerte de sécurité, il est important de se baser sur les fichiers `package-lock.json` ou équivalent, de sorte à vérifier la version des librairies effectivement résolue par le gestionnaire de packages.

! Retex Sha1-Hulud 2.0 (fin novembre 2025)

Fin novembre 2025, le CERT-FR publie l'actualité [CERTFR-2025-ACT-051](#) faisant état d'une "attaque par la chaîne d'approvisionnement de plusieurs paquets NPM".

Sujet également documenté sur le site de Microsoft : <https://www.microsoft.com/en-us/security/blog/2025/12/09/shai-hulud-2-0-guidance-for-detecting-investigating-and-defending-against-the-supply-chain-attack/>.

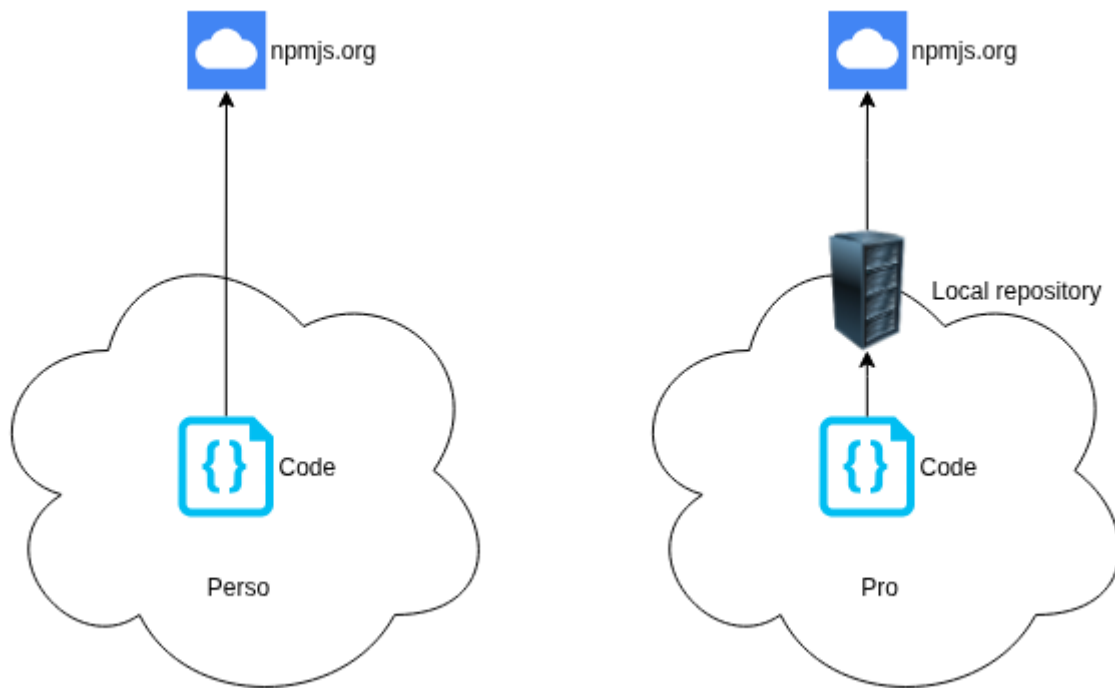
Pour faciliter les analyses d'impact, on a pu se baser sur un outil disponible sur GitHub : <https://github.com/gensecaiHQ/Shai-Hulud-2.0-Detector>.

En observant la [liste des fichiers supportés](#) par Shai-Hulud 2.0 Detector, on peut voir que cet outil considère les fichiers suivant pour mener l'analyse :

- `package.json`,
- `package-lock.json`,
- `yarn.lock`,
- `npm-shrinkwrap.json`,
- `pnpm-lock.yaml`.

2.3.5. Utilisation d'un repo local

Les grandes entreprises déploient généralement un serveur intermédiaire entre les registries officielles sur Internet et le SI de l'entreprise.



On trouve plusieurs noms possibles pour ce type de serveur intermédiaire :

- Binary Repository Manager,
- Artifact Repository,
- Package Repository,
- Software Repository,
- ...

Ce type d'outil permet l'enregistrement de fichiers binaires potentiellement volumineux, tels que :

- des programmes d'installation,
- des images de conteneurs ou de machines virtuelles,
- des résultats de compilations,
- des données de test,
- des configurations,
- des archives,
- ...

Ces outils peuvent également être configurés en relais de serveurs de packages.

Ce type de déploiement peut présenter les avantages suivants :

- accélération des téléchargements de packages, par la constitution d'un cache local, et l'apport de résilience en cas de perturbations réseaux avec Internet ;
- indépendance pour la maintenabilité à long terme : il peut arriver que certains packages disparaissent des registries, et on peut également anticiper que les registries risquent d'être arrêtées à l'avenir ;
- possibilité de renforts de la sécurité, avec mise en oeuvre de contrôles d'anti-virus sur les packages téléchargés depuis les registries ;
- capacités d'inventaires logiciels (SBOM), pouvant constituer une réponse aux questions juridiques et de sécurité évoquées précédemment.

2.4. Build

2.4.1. Compilateurs / linkers

2.4.2. Bundlers

2.4.3. Gestionnaires de projets

2.5. Industrialisation

La constitution d'images Docker embarquant des versions d'outils bien identifiées contribue à la maîtrise de l'environnement de développement.

Grâce à ces images, on peut répéter facilement une configuration de développement, pour chaque membre de l'équipe, ainsi que pour la chaîne de CI/CD, et avec l'assurance d'utiliser les mêmes versions des outils.

3. Exécution

Dès les premières versions de notre logiciel disponibles, il s'avère intéressant de savoir exécuter notre production aux différentes étapes de sa construction :

- en cours de développement, pour des tests unitaires, ou tests d'intégration libres,
- sur des versions intermédiaires, pour des tests de non-régression quotidiens par exemple,
- sur des versions identifiées, pour des campagnes de test avant livraison.

4. Assurance qualité

4.1. Pratiques d'équipe

4.2. Règles de codage

4.3. Versionning / Gestion de configuration

4.3.1. Source control - git

4.3.2. Versioning - SemVer

4.3.3. Ticketing

4.3.4. PLM - Product Lifecicle Management

4.3.5. Archivage

4.3.6. Index de configuration

4.4. Documentation

4.4.1. GED

4.4.2. CMS

4.4.3. Formats texte + git

5. Delivery

5.1. Enregistrements

5.2. CI/CD

6. Déploiement
